

Image Classification with the MNIST Dataset

In traditional programming, the programmer is able to articulate rules and conditions in their code that their program can then use to act in the correct way. This approach continues to work exceptionally well for a huge variety of problems.

Image classification, which asks a program to correctly classify an image it has never seen before into its correct class, is near impossible to solve with traditional programming techniques. How could a programmer possibly define the rules and conditions to correctly classify a huge variety of images, especially taking into account images that they have never seen?

The Solution: Deep Learning

Deep learning excels at pattern recognition by trial and error. By training a deep neural network with sufficient data, and providing the network with feedback on its performance via training, the network can identify, though a huge amount of iteration, its own set of conditions by which it can act in the correct way.

The MNIST Dataset

In the history of deep learning, the accurate image classification of the [MNIST dataset](#), a collection of 70,000 grayscale images of handwritten digits from 0 to 9, was a major development. While today the problem is considered trivial, doing image classification with MNIST has become a kind of "Hello World" for deep learning.

Training and Validation Data and Labels

When working with images for deep learning, we need both the images themselves, usually denoted as X , and also, correct [labels](#) for these images, usually denoted as Y . Furthermore, we need X and Y values both for *training* the model, and then, a separate set of X and Y values for *validating* the performance of the model after it has been trained. Therefore, we need 4 segments of data for the MNIST dataset:

1. x_{train} : Images used for training the neural network
2. y_{train} : Correct labels for the x_{train} images, used to evaluate the model's predictions during training
3. x_{valid} : Images set aside for validating the performance of the model after it has been trained
4. y_{valid} : Correct labels for the x_{valid} images, used to evaluate the model's predictions after it has been trained

The process of preparing data for analysis is called [Data Engineering](#). To learn more about the differences between training data and validation data (as well as test data), check out [this article](#) by Jason Brownlee.

Difference between GPU and TPU

GPUs are made for parallel processing, ideal for training complex neural networks. TPUs take this specialization further, focusing on tensor operations to achieve higher speeds and energy efficiencies.

Tensors are mathematical objects from linear algebra and are used to represent multidimensional objects. They can be used to perform the same arithmetic operations that are already familiar with vectors or matrices, for example.

Loading the Data Into Memory (with Keras)

There are many [deep learning frameworks](#), each with their own merits. We will be working with [Tensorflow 2](#), and specifically with the [Keras API](#). Keras has many useful built in functions designed for the computer vision tasks. It is also a legitimate choice for deep learning in a professional setting due to its [readability](#) and efficiency, though it is not alone in this regard, and it is worth investigating a variety of frameworks when beginning a deep learning project.

One of the many helpful features that Keras provides are modules containing many helper methods for [many common datasets](#), including MNIST.

We will begin by loading the Keras dataset module for MNIST:

```
In [1]: from tensorflow.keras.datasets import mnist
```

WARNING:tensorflow:From C:\Users\pdsin\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\losses.py:2976: The name tf.losses.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

With the `mnist` module, we can easily load the MNIST data, already partitioned into images and labels for both training and validation:

```
In [2]: # the data, split between train and validation sets
(x_train, y_train), (x_valid, y_valid) = mnist.load_data()
```

Exploring the MNIST Data

We stated above that the MNIST dataset contained 70,000 grayscale images of handwritten digits. By executing the following cells, we can see that Keras has partitioned 60,000 of these images for training, and 10,000 for validation (after training), and also, that each image itself is a 2D array with the dimensions 28x28:

```
In [5]: x_train.shape
```

```
Out[5]: (60000, 28, 28)
```

```
In [6]: x_valid.shape
```

```
Out[6]: (10000, 28, 28)
```

Furthermore, we can see that these 28x28 images are represented as a collection of unsigned 8-bit integer values between 0 and 255, the values corresponding with a pixel's grayscale value where 0 is black, 255 is white, and all other values are in between:

```
In [7]: x_train.dtype
```

```
Out[7]: dtype('uint8')
```

```
In [8]: x_train.min()
```

Out[8]: 0

```
In [9]: x_train.max()
```

```
Out[9]: 255
```

```
In [10]: x_train[0]
```

```
Out[10]: array([[ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0],
 [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0],
 [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0],
 [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0],
 [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
  0,  0],
 [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 3, 18, 18, 18, 126, 136, 175, 26, 166, 255, 247, 127, 0, 0,
 0,  0],
 [ 0,  0,  0,  0,  0,  0,  0,  0, 30, 36, 94, 154, 170,
253, 253, 253, 253, 253, 225, 172, 253, 242, 195, 64, 0, 0,
0,  0],
 [ 0,  0,  0,  0,  0,  0,  0, 49, 238, 253, 253, 253, 253,
253, 253, 253, 251, 93, 82, 82, 56, 39, 0, 0, 0, 0,
0,  0],
 [ 0,  0,  0,  0,  0,  0,  0, 18, 219, 253, 253, 253, 253,
253, 198, 182, 247, 241, 0, 0, 0, 0, 0, 0, 0, 0,
0,  0],
 [ 0,  0,  0,  0,  0,  0,  0,  0, 80, 156, 107, 253, 253,
205, 11, 0, 43, 154, 0, 0, 0, 0, 0, 0, 0, 0,
0,  0],
 [ 0,  0,  0,  0,  0,  0,  0,  0,  0, 14, 1, 154, 253,
90, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,  0],
 [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 139, 253,
190, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,  0],
 [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 11, 190,
253, 70, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0,  0],
 [ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 35,
```

```

241, 225, 160, 108, 1, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
81, 240, 253, 253, 119, 25, 0, 0, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 45, 186, 253, 253, 150, 27, 0, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 16, 93, 252, 253, 187, 0, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 249, 253, 249, 64, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 46, 130, 183, 253, 253, 207, 2, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 39,
148, 229, 253, 253, 253, 250, 182, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 24, 114, 221,
253, 253, 253, 253, 201, 78, 0, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 23, 66, 213, 253, 253,
253, 253, 198, 81, 2, 0, 0, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 18, 171, 219, 253, 253, 253, 253,
195, 80, 9, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 55, 172, 226, 253, 253, 253, 253, 244, 133,
11, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 136, 253, 253, 253, 212, 135, 132, 16, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0],
[ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0]], dtype=uint8)

```

Using [Matplotlib](#), we can render one of these grayscale images in our dataset:

```

In [15]: import matplotlib.pyplot as plt

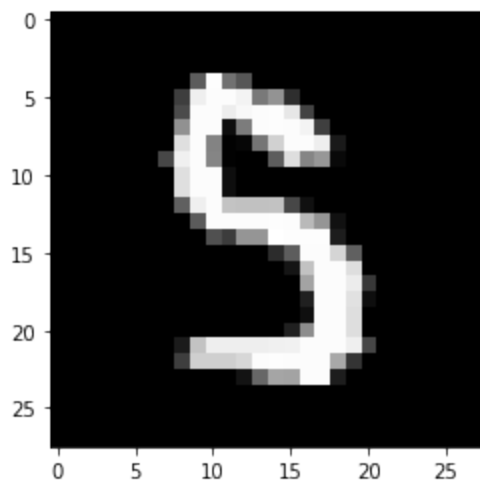
         image = x_train[748]
         plt.imshow(image, cmap='gray')

```

```

Out[15]: <matplotlib.image.AxesImage at 0x260f99b3310>

```



In this way we can now see that this is a 28x28 pixel image of a 5. Or is it a 3? The answer is in the `y_train` data, which contains correct labels for the data. Let's take a look:

```
In [16]: y_train[748]
```

```
Out[16]: 5
```

Preparing the Data for Training

In deep learning, it is common that data needs to be transformed to be in the ideal state for training. For this particular image classification problem, there are 3 tasks we should perform with the data in preparation for training:

1. Flatten the image data, to simplify the image input into the model
2. Normalize the image data, to make the image input values easier to work with for the model
3. Categorize the labels, to make the label values easier to work with for the model

Flattening the Image Data

Though it's possible for a deep learning model to accept a 2-dimensional image (in our case 28x28 pixels), we're going to simplify things to start and [reshape](#) each image into a single array of 784 continuous pixels (note: $28 \times 28 = 784$). This is also called flattening the image.

Here we accomplish this using the helper method `reshape` :

```
In [17]: x_train = x_train.reshape(60000, 784)
         x_valid = x_valid.reshape(10000, 784)
```

We can confirm that the image data has been reshaped and is now a collection of 1D arrays containing 784 pixel values each:

```
In [18]: x_train.shape
```

```
Out[18]: (60000, 784)
```

```
In [19]: x_train[0]
```

```

Out[19]: array([ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  3, 18, 18, 18,
126, 136, 175, 26, 166, 255, 247, 127,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0, 30, 36, 94, 154, 170, 253,
253, 253, 253, 253, 225, 172, 253, 242, 195, 64,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0, 49, 238, 253, 253, 253,
253, 253, 253, 253, 253, 251, 93, 82, 82, 56, 39,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 18, 219, 253,
253, 253, 253, 253, 198, 182, 247, 241,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
80, 156, 107, 253, 253, 205, 11,  0, 43, 154,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0, 14,  1, 154, 253, 90,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0, 139, 253, 190,  2,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 11, 190, 253, 70,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 35,
241, 225, 160, 108,  1,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0, 81, 240, 253, 253, 119, 25,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0, 45, 186, 253, 253, 150, 27,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0, 16, 93, 252, 253, 187,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 249,
253, 249, 64,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 46, 130,
183, 253, 253, 207,  2,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 39, 148,
229, 253, 253, 253, 250, 182,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 24, 114,
221, 253, 253, 253, 253, 201, 78,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 23, 66,
213, 253, 253, 253, 253, 198, 81,  2,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 18, 171,
219, 253, 253, 253, 253, 195, 80,  9,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 55, 172,
226, 253, 253, 253, 253, 244, 133, 11,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
136, 253, 253, 253, 212, 135, 132, 16,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0,
 0,  0,  0,  0], dtype=uint8)

```

Normalizing the Image Data

Deep learning models are better at dealing with floating point numbers between 0 and 1. Converting integer values to floating point values between 0 and 1 is called [normalization](#), and a simple approach we will take here to normalize the data will be to divide all the pixel values (which if you recall are between 0 and 255) by 255:

```
In [20]: x_train = x_train / 255
         x_valid = x_valid / 255
```

We can now see that the values are all floating point values between 0.0 and 1.0 :

```
In [21]: x_train.dtype
```

```
Out[21]: dtype('float64')
```

```
In [22]: x_train.min()
```

```
Out[22]: 0.0
```

```
In [23]: x_train.max()
```

```
Out[23]: 1.0
```

Categorical Encoding

Consider for a moment, if we were to ask, what is 7 - 2? Stating that the answer was 4 is closer than stating that the answer was 9. However, for this image classification problem, we don't want the neural network to learn this kind of reasoning: we just want it to select the correct category, and understand that if we have an image of the number 5, that guessing 4 is just as bad as guessing 9.

As it stands, the labels for the images are integers between 0 and 9. Because these values represent a numerical range, the model might try to draw some conclusions about its performance based on how close to the correct numerical category it guesses.

Therefore, we will do something to our data called categorical encoding. This kind of transformation modifies the data so that each value is a collection of all possible categories, with the actual category that this particular value is set as true.

As a simple example, consider if we had 3 categories: red, blue, and green. For a given color, 2 of these categories would be false, and the other would be true:

Actual Color	Is Red?	Is Blue?	Is Green?
Red	True	False	False
Green	False	False	True
Blue	False	True	False
Green	False	False	True

Rather than use "True" or "False", we could represent the same using binary, either 0 or 1:

Actual Color	Is Red?	Is Blue?	Is Green?
Red	1	0	0
Green	0	0	1
Blue	0	1	0
Green	0	0	1

This is what categorical encoding is, transforming values which are intended to be understood as categorical labels into a representation that makes their categorical nature explicit to the model. Thus, if we were using these values for training, we would convert...

```
values = ['red', 'green', 'blue', 'green']
```

... which a neural network would have a very difficult time making sense of, instead to:

```
values = [
    [1, 0, 0],
    [0, 0, 1],
    [0, 1, 0],
    [0, 0, 1]
]
```

Categorically Encoding the Labels

Keras provides a utility to [categorically encode values](#), and here we use it to perform categorical encoding for both the training and validation labels:

```
In [24]: import tensorflow.keras as keras
num_categories = 10

y_train = keras.utils.to_categorical(y_train, num_categories)
y_valid = keras.utils.to_categorical(y_valid, num_categories)
```

Here are the first 10 values of the training labels, which you can see have now been categorically encoded:

```
In [25]: y_train[0:9]
```

```
Out[25]: array([[0., 0., 0., 0., 0., 1., 0., 0., 0., 0.],
 [1., 0., 0., 0., 0., 0., 0., 0., 0., 0.],
 [0., 0., 0., 0., 1., 0., 0., 0., 0., 0.],
 [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
 [0., 0., 0., 0., 0., 0., 0., 0., 0., 1.],
 [0., 0., 1., 0., 0., 0., 0., 0., 0., 0.],
 [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.],
 [0., 0., 0., 1., 0., 0., 0., 0., 0., 0.],
 [0., 1., 0., 0., 0., 0., 0., 0., 0., 0.]], dtype=float32)
```

Creating the Model

With the data prepared for training, it is now time to create the model that we will train with the data. This first basic model will be made up of several *layers* and will be comprised of 3 main parts:

1. An input layer, which will receive data in some expected format
2. Several [hidden layers](#), each comprised of many *neurons*. Each [neuron](#) will have the ability to affect the network's guess with its *weights*, which are values that will be updated over many iterations as the network gets feedback on its performance and learns
3. An output layer, which will depict the network's guess for a given image

Instantiating the Model

To begin, we will use Keras's [Sequential](#) model class to instantiate an instance of a model that will have a series of layers that data will pass through in sequence:

```
In [26]: from tensorflow.keras.models import Sequential

        model = Sequential()
```

WARNING:tensorflow:From C:\Users\pdsin\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\backend.py:873: The name tf.get_default_graph is deprecated. Please use tf.compat.v1.get_default_graph instead.

Creating the Input Layer

Next, we will add the input layer. This layer will be *densely connected*, meaning that each neuron in it, and its weights, will affect every neuron in the next layer. To do this with Keras, we use Keras's [Dense](#) layer class.

```
In [27]: from tensorflow.keras.layers import Dense
```

The `units` argument specifies the number of neurons in the layer. We are going to use `512` (chosen from experimentation). Choosing the correct number of neurons is what puts the "science" in "data science" as it is a matter of capturing the statistical complexity of the dataset. Try playing around with this value later to see how it affects training and to start developing a sense for what this number means.

We will learn more about activation functions later, but for now, we will use the `relu` activation function, which in short, will help our network to learn how to make more sophisticated guesses about data than if it were required to make guesses based on some strictly linear function.

The `input_shape` value specifies the shape of the incoming data which in our situation is a 1D array of 784 values:

```
In [28]: model.add(Dense(units=512, activation='relu', input_shape=(784,)))
```

Creating the Hidden Layer

Now we will add an additional densely connected layer. Again, much more will be said about these later, but for now know that these layers give the network more parameters to contribute towards its guesses, and therefore, more subtle opportunities for accurate learning:

```
In [29]: model.add(Dense(units = 512, activation='relu'))
```

Creating the Output Layer

Finally, we will add an output layer. This layer uses the activation function `softmax` which will result in each of the layer's values being a probability between 0 and 1 and will result in all the outputs of the layer adding to 1. In this case, since the network is to make a guess about a single image belonging to 1 of 10 possible categories, there will be 10 outputs. Each output gives the model's guess (a probability) that the image belongs to that specific class:

```
In [30]: model.add(Dense(units = 10, activation='softmax'))
```

Summarizing the Model

Keras provides the model instance method `summary` which will print a readable summary of a model:

```
In [32]: model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
=====	=====	=====
dense (Dense)	(None, 512)	401920
dense_1 (Dense)	(None, 512)	262656
dense_2 (Dense)	(None, 10)	5130
=====	=====	=====
Total params: 669706 (2.55 MB)		
Trainable params: 669706 (2.55 MB)		
Non-trainable params: 0 (0.00 Byte)		

Note the number of trainable parameters. Each of these can be adjusted during training and will contribute towards the trained model's guesses.

Compiling the Model

Again, more details are to follow, but the final step we need to do before we can actually train our model with data is to `compile` it. Here we specify a `loss function` which will be used for the model to understand how well it is performing during training. We also specify that we would like to track `accuracy` while the model trains:

```
In [33]: model.compile(loss='categorical_crossentropy', metrics=['accuracy'])
```

```
WARNING:tensorflow:From C:\Users\pdsin\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\optimizers\__init__.py:309: The name tf.train.Optimizer is deprecated. Please use tf.compat.v1.train.Optimizer instead.
```

Training the Model

Now that we have prepared training and validation data, and a model, it's time to train our model with our training data, and verify it with its validation data.

"Training a model with data" is often also called "fitting a model to data." Put this latter way, it highlights that the shape of the model changes over time to more accurately understand the data that it is being given.

When fitting (training) a model with Keras, we use the model's `fit` method. It expects the following arguments:

- The training data
- The labels for the training data
- The number of times it should train on the entire training dataset (called an *epoch*)
- The validation or test data, and its labels

Run the cell below to train the model. We will discuss its output after the training completes:

```
In [34]: history = model.fit(
          x_train, y_train, epochs=5, verbose=1, validation_data=(x_valid, y_valid)
        )
```

Epoch 1/5

WARNING:tensorflow:From C:\Users\pdsin\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\utils\tf_utils.py:492: The name tf.ragged.RaggedTensorValue is deprecated. Please use tf.compat.v1.ragged.RaggedTensorValue instead.

WARNING:tensorflow:From C:\Users\pdsin\AppData\Local\Programs\Python\Python310\lib\site-packages\keras\src\engine\base_layer_utils.py:384: The name tf.executing_eagerly_outside_functions is deprecated. Please use tf.compat.v1.executing_eagerly_outside_functions instead.

1875/1875 [=====] - 16s 8ms/step - loss: 0.1875 - accuracy: 0.9443 - val_loss: 0.1027 - val_accuracy: 0.9691

Epoch 2/5

1875/1875 [=====] - 14s 8ms/step - loss: 0.0855 - accuracy: 0.9757 - val_loss: 0.0933 - val_accuracy: 0.9743

Epoch 3/5

1875/1875 [=====] - 15s 8ms/step - loss: 0.0604 - accuracy: 0.9830 - val_loss: 0.0718 - val_accuracy: 0.9813

Epoch 4/5

1875/1875 [=====] - 15s 8ms/step - loss: 0.0461 - accuracy: 0.9875 - val_loss: 0.0878 - val_accuracy: 0.9782

Epoch 5/5

1875/1875 [=====] - 14s 7ms/step - loss: 0.0373 - accuracy: 0.9895 - val_loss: 0.0918 - val_accuracy: 0.9803

Observing Accuracy

For each of the 5 epochs, notice the `accuracy` and `val_accuracy` scores. `accuracy` states how well the model did for the epoch on all the training data. `val_accuracy` states how well the model did on the validation data, which if you recall, was not used at all for training the model.

The model did quite well! The accuracy quickly reached close to 100%, as did the validation accuracy. We now have a model that can be used to accurately detect and classify hand-written images.

The next step would be to use this model to classify new not-yet-seen handwritten images. This is called [inference](#).

What a Neuron Does

Ultimately, each neuron is trying to fit a line to some data. Below, we have some datapoints and a randomly drawn line using the equation $y = mx + b$.

Try changing the m and the b in order to find the lowest possible loss. How did you find the best line? Can you make a program to follow your strategy?

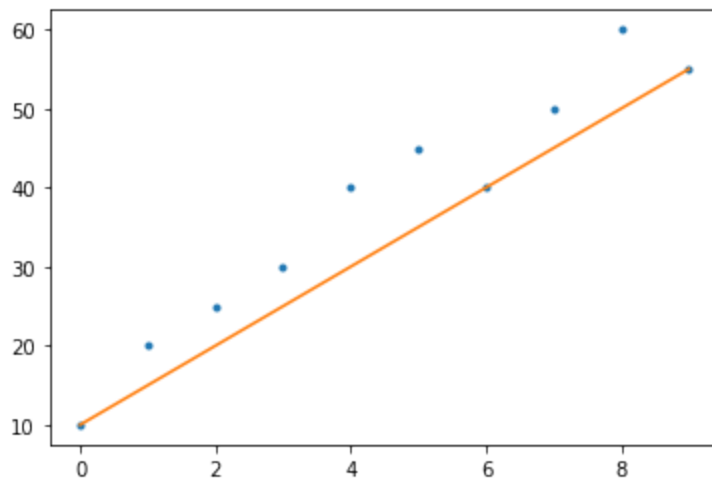
```
In [36]: import numpy as np
from numpy.polynomial.polynomial import polyfit
import matplotlib.pyplot as plt

m = 5 # -2 to start, change me please
b = 10 # 40 to start, change me please

# Sample data
x = np.array([ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
y = np.array([10, 20, 25, 30, 40, 45, 40, 50, 60, 55])
y_hat = x * m + b

plt.plot(x, y, '.')
plt.plot(x, y_hat, '-')
plt.show()

print("Loss:", np.sum((y - y_hat)**2)/len(x))
```



Loss: 40.0

In []: